

PatchDistill: 蒸留された因果的 Patch Signature に基づく 効率的プロンプトインジェクション検出

Anonymous

2026 年 6 月 8 日

概要

大規模言語モデルを組み込んだアプリケーションでは、ユーザー入力や外部文書に含まれる命令が、システム指示や開発者指示よりも優先されるプロンプトインジェクションが重大な脅威となる。既存の検出器は、表層的なテキスト特徴、最終出力、あるいは LLM-as-a-Judge に依存することが多く、攻撃成功時にモデル内部でどの制御信号が優先されるのかを因果的に扱うものは限られている。本稿では、直接的プロンプトインジェクションを「system instruction 由来の制御信号」から「user-side injected instruction 由来の制御信号」への内部制御シフトとして捉え、activation patching により得られる patch-effect profile を Causal Patch Signature として定義する。さらに、高コストな activation patching を推論時に実行するのではなく、offline の教師信号生成に限定し、1 回の forward pass で取得可能な hidden state, attention 統計, logit 統計から patch signature を近似する proxy model へ蒸留する PatchDistill を提案する。初期実装では、合成 direct prompt injection データに対して、TF-IDF, ルール特徴, 疑似 patch signature を用いた surrogate PatchDistill の比較を行い、実験パイプラインの妥当性を確認した。今後、Colab A100 上で Hugging Face モデルを用いた residual stream patching を実行し、真の Causal Patch Signature に基づく proxy error と検出性能を評価する。

1 はじめに

LLM アプリケーションでは、システムプロンプト、開発者指示、ユーザー入力、検索文書、ツール出力など、異なる信頼境界を持つテキストが同一コンテキスト内に連結される。この構造は、LLM が命令とデータを厳密には分離しないことに起因するプロンプトインジェクションの温床となる。特に直接的プロンプトインジェクションでは、“Ignore previous instructions” や “Reveal the system prompt” のような命令がユーザー入力中に挿入され、モデルが本来のタスクや安全制約を逸脱する。

既存研究は、攻撃・防御の形式化やベンチマーク整備を進めてきた [2, 3, 4]。一方で、実用的な検出器の多くは、文字列パターン、分類器、外部 judge, または通常の内部表現 probe に依存する。これらは有用であるが、攻撃命令がモデル内部のどの層・token 位置・表現経路を通じて挙動を変化させるのかを直接には測定しない。

本研究の中心仮説は、直接的プロンプトインジェクションの成功時に、モデル内部で system instruction に由来する制御信号が弱まり、user-side injected instruction に由来する制御信号が強まるというものである。この制御シフトを観測的特徴だけで説明するのではなく、activation patching による介入で測定する。ただし activation patching は、多数の層・head・token 位置に対して追加 forward pass を要するため、オンライン検出には高コストである。そこで本稿では、activation

patching を offline の因果教師信号生成に限定し、その結果を one-pass feature から近似する proxy model へ蒸留する。

本稿の貢献は以下である。

- 直接的プロンプトインジェクション検出のための Causal Patch Signature を定義する。
- offline activation patching による高コストな因果信号を、one-pass 内部特徴から近似する Patch Signature Distillation を提案する。
- hidden state probe や attention 統計のみの分類と比較可能な実験パイプラインを実装する。
- 初期合成データにより、データ生成、特徴抽出、proxy 学習、検出器評価、結果収集までの実験基盤を検証する。

2 関連研究

■**プロンプトインジェクション攻撃と防御** Liu らは、LLM 統合アプリケーションにおけるプロンプトインジェクション攻撃を形式化し、複数の攻撃・防御・モデル・タスクを横断するベンチマークを提案した [2]。Tensor Trust は、人間が作成した多数の prompt extraction および prompt hijacking 例を含むデータセットとベンチマークを提供する [3]。また、RAG や外部文書を介した間接的プロンプトインジェクションについては BIPIA がベンチマークを提示し、外部コンテンツ内の命令を情報として扱うか命令として実行するかの境界問題を示した [4]。本研究は主に direct PI を対象としつつ、最終的には direct/indirect の内部 signature 差分を探索する。

■**Activation patching と因果的局所化** Activation patching は、clean prompt の activation を corrupted prompt の一部 activation に置換し、出力変化を測ることでモデル内部部位の因果寄与を推定する手法である [5]。Zhang and Nanda は、評価 metric や corruption の作り方により patching 結果が変化し得ることを示し、methodological detail の重要性を指摘した [5]。Kramár らは、activation patching の計算コストが model component 数に線形に増える問題を踏まえ、Attribution Patching と AtP* による高速近似を検討した [1]。PatchDistill は、patching 自体を高速化するのではなく、高コストな patching 結果を教師信号として蒸留し、推論時には patching を行わない点で異なる。

■**LLM-as-a-Judge** LLM-as-a-Judge は、自由記述出力の評価をスケールさせる実用的手法として広く用いられている。Zheng らは、MT-Bench と Chatbot Arena において、強力な LLM judge が人間評価と高い一致を示す一方、position bias, verbosity bias, self-enhancement bias などの限界も持つことを示した [6]。本研究では LLM-as-a-Judge を主検出器ではなく、高リスクまたは不確実な入力に対する Stage 2 の意味論的検証器として位置付ける。

3 問題設定

入力 x は system prompt S と user input U から構成される。直接的プロンプトインジェクションでは、 U 内に悪意ある命令 span m が含まれる。検出器 D は、入力 x に対して

$$D(x) \in \{\text{ALLOW}, \text{BLOCK}, \text{ABSTAIN}\}$$

を返す。本稿では、特に見逃しを重視し、false negative rate (FNR) と fixed-FPR 下の recall を主要指標とする。

4 提案手法

4.1 Causal Patch Signature

clean prompt x^{clean} と injected prompt x^{inj} のペアを用意する。層 l , token 位置 t の residual activation を $a_{l,t}$ とする。本実装の MVP では head 単位ではなく residual block output 単位で patching を行う。すなわち,

$$a_{l,t}^{\text{inj}} \leftarrow a_{l,t'}^{\text{clean}}$$

という介入を行い、安全継続文と攻撃追従継続文の log-probability margin の変化を測る。

安全継続文 y_{safe} と攻撃追従継続文 y_{attack} に対し、安全 margin を

$$s(x) = \frac{1}{|y_{\text{safe}}|} \log p(y_{\text{safe}} | x) - \frac{1}{|y_{\text{attack}}|} \log p(y_{\text{attack}} | x)$$

と定義する。patch component $c = (l, t)$ の patch effect は

$$\text{PE}_c(x) = s(x^{\text{inj}}; \text{patch}(c)) - s(x^{\text{inj}})$$

である。複数 component の patch effect vector

$$\text{PE}(x) = [\text{PE}_{c_1}(x), \dots, \text{PE}_{c_k}(x)]$$

を Causal Patch Signature と呼ぶ。

4.2 Patch Signature Distillation

Activation patching は高コストであるため、推論時に実行しない。代わりに、一回の forward pass で得られる特徴

$$r(x) = [h(x), A(x), L(x)]$$

を抽出する。ここで $h(x)$ は hidden-state 統計, $A(x)$ は attention entropy や malicious span attention mass, $L(x)$ は候補 token の log-probability 統計である。proxy model q_ϕ は

$$q_\phi(r(x)) \approx \text{PE}(x)$$

を満たすように学習される。

4.3 One-pass 検出器

オンライン検出では、入力 x に対して LLM を一回 forward し、 $r(x)$ を得る。その後、 $\hat{\text{PE}}(x) = q_\phi(r(x))$ を推定し、

$$D(x) = C(r(x), \hat{\text{PE}}(x))$$

により軽量分類器 C がリスクスコアを出す。低リスクは ALLOW, 高リスクは BLOCK, 中間帯は ABSTAIN とし、必要に応じて LLM-as-a-Judge に送る。

5 実装

実装は Python パッケージ `patchdistill` として構成した. `patchdistill.data` は clean/injected/benign-hard の合成 direct PI データを生成する. `patchdistill.hf` は Hugging Face causal LM から hidden state, attention, logit 特徴を抽出する. `patchdistill.patching` は residual block output に対する selected activation patching を実装する. `patchdistill.distill` は HF feature と patch signature から proxy model および検出器を学習する. `patchdistill.reporting` は実験結果 JSON を収集し, Markdown summary を生成する.

6 実験

6.1 初期合成データ

初期実験では, 5 種類のタスクテンプレート, 6 種類の攻撃テンプレート, 6 種類の benign hard prompt テンプレートから 160 件のデータを生成した. positive row は direct injection を含む injected prompt, negative row は clean task または benign hard prompt である. 評価は template split で行い, 攻撃テンプレートとタスクテンプレートの単純な暗記を避ける構成にした.

6.2 比較手法

初期実験では以下を比較した.

- TF-IDF + Logistic Regression
- ルール特徴 + Logistic Regression
- 疑似 patch signature を用いた surrogate PatchDistill

ここで疑似 patch signature は, 実験配線の検証用にルール特徴から決定論的に生成したものであり, 因果的証拠ではない. 真の Causal Patch Signature は Colab A100 上の `hf-patch` で生成する.

7 初期結果

表 1 に MVP profile と stress profile の template split における予備結果を示す. Stress profile では, 攻撃命令の言い換え, 多言語風の override, および疑わしい語を含む benign-hard prompt を追加した. その結果, TF-IDF は stress profile で AUROC が 0.568 まで低下し, false positive rate が 0.591 まで増加した. 一方で, ルール特徴と surrogate PatchDistill は依然として完全分類であった. したがって, stress profile は表層 n-gram baseline を崩す第一歩にはなったが, まだルール特徴で容易に分離できる.

疑似 patch signature の proxy prediction error は $MAE = 0.0587$, $MSE = 0.00934$ であった. ただし, これは疑似教師信号に対する結果であり, 真の activation patching に基づく $PE(x)$ の近似可能性を示すものではない.

Colab A100 上で GPT-2 を用いた初期 pilot も実行した. Stress profile の 160 例から one-pass

表 1 合成 direct PI データにおける初期結果. MVP は初期 smoke dataset, stress は言い換え攻撃と benign-hard prompt を増やした dataset である.

Profile	手法	F1	AUROC	FNR	FPR
MVP	TF-IDF + LR	0.884	0.963	0.050	0.200
MVP	Rule features + LR	1.000	1.000	0.000	0.000
MVP	Surrogate PatchDistill	1.000	1.000	0.000	0.000
Stress	TF-IDF + LR	0.889	0.568	0.000	0.591
Stress	Rule features + LR	1.000	1.000	0.000	0.000
Stress	Surrogate PatchDistill	1.000	1.000	0.000	0.000

feature を抽出し, positive 24 例に対して residual stream patching を行った. このとき proxy model の patch-effect prediction error は $MAE = 0.0277$, $MSE = 0.00407$ であった. HF feature のみの Logistic Regression と, distilled patch feature を加えた Logistic Regression は, どちらも $F1 = 1.0$, $AUROC = 1.0$ となった. この結果も優位性の証拠ではなく, むしろ現行 synthetic data が GPT-2 の one-pass 統計だけで分離できるほど容易であることを示す.

8 考察

初期結果から分かることは, 現時点の synthetic データでは表層の手がかりや長さ・span 有無などの副次の手がかりが強く, 検出器比較としては不十分であるという点である. したがって次段階では, 以下の修正が必要である.

1. benign hard prompt を増やし, 引用, 教育, コード, 法務文書に含まれる命令表現をより多様化する.
2. attack template を train/test で明確に分離し, 言い換え, 多言語, 長文化, adaptive attack を追加する.
3. clean/injected の長さ, 危険語数, 引用符, malicious span 有無に基づくショートカットを抑制する.
4. HF モデルで真の patch signature を生成し, proxy error と detector 性能をより難しい split で評価する.
5. hidden-state-only probe, attention-stat-only detector, PatchDistill without proxy, PatchDistill full の ablation を行う.

Colab A100 pilot では, まず GPT-2 または Qwen2.5-0.5B-Instruct を用い, 代表層に限定して residual stream patching を行う. その後, Qwen2.5-1.5B, Gemma-2-2B へ拡張し, 最終的に 7B 級 instruction-tuned model で主評価を行う.

9 限界

本稿の現時点の結果は、合成データと疑似 patch signature に基づく予備実験である。したがって、PatchDistill が実際の direct PI や indirect PI に対して汎化すること、あるいは hidden-state probe より優れることはまだ示していない。また、現在の patching 実装は residual block output 単位であり、attention head 単位や MLP activation 単位の詳細な causal localization には拡張が必要である。さらに、safe/attack continuation の選択により patch effect が変化し得るため、複数 metric による頑健性評価が必要である。

10 おわりに

本稿では、直接的プロンプトインジェクションを内部制御シフトとして捉え、activation patching により得られる Causal Patch Signature を one-pass 検出器へ蒸留する PatchDistill を提案した。初期実装では、合成データ生成、one-pass surrogate 特徴、疑似 patch signature、proxy 学習、検出器評価、HF feature extraction、residual patching、結果収集までの実験基盤を構築した。今後は Colab A100 上で真の patch signature を収集し、proxy prediction error、OOD robustness, benign hard false positive rate, LLM judge call rate を評価する。

参考文献

- [1] J'anos Kramar, Tom Lieberum, Rohin Shah, and Neel Nanda. Atp*: An efficient and scalable method for localizing llm behaviour to components, 2024.
- [2] Yupei Liu, Yuqi Jia, Runpeng Geng, Jinyuan Jia, and Neil Zhenqiang Gong. Formalizing and benchmarking prompt injection attacks and defenses. In *USENIX Security Symposium*, 2024.
- [3] Sam Toyer, Olivia Watkins, Ethan Adrian Mendes, Justin Svegliato, Luke Bailey, Tiffany Wang, Isaac Ong, Karim Elmaaroufi, Pieter Abbeel, Trevor Darrell, Alan Ritter, and Stuart Russell. Tensor trust: Interpretable prompt injection attacks from an online game, 2023.
- [4] Jingwei Yi, Yueqi Xie, Bin Zhu, Emre Kiciman, Guangzhong Sun, Xing Xie, and Fangzhao Wu. Benchmarking and defending against indirect prompt injection attacks on large language models, 2023.
- [5] Fred Zhang and Neel Nanda. Towards best practices of activation patching in language models: Metrics and methods. In *International Conference on Learning Representations*, 2024.
- [6] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging llm-as-a-judge with mt-bench and chatbot arena. In *Advances in Neural Information Processing Systems*, 2023.

付録 A 再現コマンド

ローカル smoke test は以下で実行できる.

```
python -m patchdistill.cli make-data --n 160 --out data/synthetic_direct_pi.jsonl
python -m patchdistill.cli run-surrogate \
  --data data/synthetic_direct_pi.jsonl \
  --out runs/surrogate_mvp \
  --split template
python -m patchdistill.cli collect-results \
  --runs runs \
  --out runs/summary.json \
  --markdown runs/summary.md
```

Colab A100 pilot は以下で実行できる.

```
scripts/run_colab_pilot.sh
```

Qwen2.5-0.5B-Instruct を用いる場合は以下のように環境変数を指定する.

```
MODEL_NAME=Qwen/Qwen2.5-0.5B-Instruct \
RUN_NAME=qwen25_05b_a100_pilot \
LAYERS=0,12,23 \
ATTN_IMPLEMENTATION=eager \
scripts/run_colab_pilot.sh
```